电子科技大学
格拉斯哥学院
Glasgow College, UESTC

Logbook

From:2025/10/01 To:2026/04/28

Month	List the main activities (only few words per activity)	Interaction with the supervisor			Any other form of supervisory interaction (second supervisor, industry, fellows etc.)
		Number of meetings	Mode of meeting (face- to-face, online e.g., Skype, WeChat etc.)	Number of emails exchanged	
October	1. PyTorch Fundamentals (10/1-10/7): - Read PyTorch technical documentation. - Learned data loading, environment configuration, and CNN concepts (convolutional/pooling layers). - Practiced processing the CIFAR dataset. 2. U-Net Deep Dive (10/9-10/17): - Systematically studied the U-Net paper. - Learned U-Net environment setup and dataset processing through tutorials. - Prepared and delivered a presentation for the group meeting on 10/17. 3. Framework Focus (10/17-10/24): - Researched U-Net variants (U-mamba, U-Net++, ResUNet). - Identified nnU-Net as an adaptive framework and decided to make it the research focus.	1	face- to-face	3	Second supervisor: Liu ZhiShuai

	- Downloaded nnU-Net v2 source code and set up a new environment. - Resolved configuration issues (Nvidia driver version, v1/v2 naming differences, 3D Nii data format). 4. Baseline Model Training & Debugging (10/25-10/26): - Followed supervisor's guidance to split the dataset (train/validation/test). - Set max epochs to 150 and started training. - Discovered that test set metrics (mIoU, Recall) were extremely low. - After a reminder from the supervisor, investigated and confirmed the issue was caused by image flipping during nnU-Net v2 data augmentation. - Obtained baseline evaluation results (mIoU: 0.8107). 5. Experimental Iterations (10/27-10/28): - (10/27) Discussed results with supervisor; retrained using the CLAHE-enhanced dataset. Results showed no significant improvement. - (10/28) Reported results; per supervisor's guidance, changed the loss function to Focal Loss and retrained. Results still showed no significant change.				

电子科技大学
格拉斯哥学院
Glasgow College, UESTC

Month	List the main activities (only few words per activity)	Interaction with the supervisor			Any other form of supervisory interaction (second supervisor, industry, fellows etc.)
		Number of meetings	Mode of meeting (face- to-face, online e.g., Skype, WeChat etc.)	Number of emails exchanged	
November	1. PA-Net Preprocessing Replication (11/01-11/07): Studied and implemented the four-step preprocessing pipeline from the PA-Net paper: Graying, Standardization, CLAHE, and Gamma Transform . Integrated these custom scripts into the nnU-Net data pipeline to enhance vessel-to-background separation . 2. Robust Strategy: 5-Fold Cross-Validation (11/08-11/15): Transitioned from a fixed dataset split to a 5-fold cross-validation strategy on the DRIVE dataset to minimize partitioning bias . Developed a batch evaluation script to automate the calculation of SE, SP, ACC, and AUC metrics. 3. Core Module Development: LPT and AFFM (11/16-11/24): Analyzed the mathematical definitions of the Lightweight Parallel Transformer (LPT) and Adaptive Vascular Feature Fusion Module (AFFM) . Wrote PyTorch code for the LFAU and EPA units within the LPT bottleneck . 4. Structural Integration & Resource Optimization (11/25-11/30): Embedded the LPT into the nnU-Net bottleneck and replaced standard skip connections with the AFFM . Optimized batch sizes to prevent "Out of Memory" (OOM) errors on the 8GB RTX 4060 hardware.	1	Face-to-face	2	

电子科技大学
格拉斯哥学院
Glasgow College, UESTC

Month	List the main activities (only few words per activity)	Interaction with the supervisor			Any other form of supervisory interaction (second supervisor, industry, fellows etc.)
		Number of meetings	Mode of meeting (face- to-face, online e.g., Skype, WeChat etc.)	Number of emails exchanged	
December	1. Experiment 5 Execution & Data Analysis (12/01-12/07): Ran full training sessions for the modified nnU-Net+LPT+AFFM architecture . Conducted a comparative analysis of quantitative results, observing an unexpected drop in Sensitivity (SE). 2. Conflict Mechanism Investigation (12/08-12/15): Investigated the structural conflict between the AFFM's multi-level fusion and nnU-Net's Deep Supervision mechanism . Hypothesized that mixed signals from the AFFM lead to "negative optimization" in the decoder layers. 3. Troubleshooting Spatial Calibration (12/16-12/22): Debugged the "180-degree rotation bug" by recalibrating the NIfTI affine transformation matrix . Verified geometric consistency by visualizing the 2D-to-3D pseudo-conversion outputs . 4. Interim Report Preparation (12/23-12/31): Aggregated all data for the final comparison tables (Table 5 & 6) and visualized results (Figures 1-5) . Outlined future work for 2026, focusing on isolated ablation studies for LPT and AFFM.	2	1 Face-to-face, 1 online	4	

电子科技大学
格拉斯哥学院
Glasgow College, UESTC

Month	List the main activities (only few words per activity)	Interaction with the supervisor			Any other form of supervisory interaction (second supervisor, industry, fellows etc.)
		Number of meetings	Mode of meeting (face- to-face, online e.g., Skype, WeChat etc.)	Number of emails exchanged	
January	1. Baseline Re-check and PA-Net Ablation (01/01-01/12): - Re-ran the nnU-Net baseline and PA-Net variants under the same DRIVE 5-fold protocol. - Compared Full PA-Net, LPT Only, AFFM Only, and LPT+AFFM settings. 2. Interaction Between AFFM and Deep Supervision (01/13-01/20): - Reviewed failed cases and metric trends from the ablation experiments. - Found that AFFM did not provide stable gains within the nnU-Net decoder structure. 3. Direction Refinement (01/21-01/31): - Chose the LPT route as the main direction for later structural improvement. - Updated experiment notes and prepared a clearer ablation plan for the final report.	2	1 Face-to-face, 1 online/WeChat	3	Technical discussion with PhD student Liu Zhishuai on PA-Net integration and ablation interpretation.

电子科技大学
格拉斯哥学院
Glasgow College, UESTC

Month	List the main activities (only few words per activity)	Interaction with the supervisor			Any other form of supervisory interaction (second supervisor, industry, fellows etc.)
		Number of meetings	Mode of meeting (face- to-face, online e.g., Skype, WeChat etc.)	Number of emails exchanged	
February	1. Deformable LPT Design (02/01-02/09): - Replaced the GAA operation in LPT with deformable sampling to better adapt to thin and curved vessels. - Implemented 4-point and 8-point deformable sampling variants. 2. Module Combination Tests (02/10-02/18): - Tested Deformable LPT without AFFM and Deformable LPT combined with AFFM. - Added modulation and multi-scale aggregation for local vessel-structure modelling. 3. Model Selection (02/19-02/28): - Compared F1, sensitivity and precision trends across the new variants. - Retained the 4-point Deformable LPT without AFFM as a more stable branch for the next stage.	2	1 Face-to-face, 1 online/WeChat	3	Informal discussion with lab peers on deformable sampling design and training stability.

电子科技大学
格拉斯哥学院
Glasgow College, UESTC

Month	List the main activities (only few words per activity)	Interaction with the supervisor			Any other form of supervisory interaction (second supervisor, industry, fellows etc.)
		Number of meetings	Mode of meeting (face- to-face, online e.g., Skype, WeChat etc.)	Number of emails exchanged	
March	1. Attention and Structural Convolution (03/01-03/10): - Added Coordinate Attention to strengthen position-aware channel weighting. - Integrated Dynamic Snake Convolution into the multi-scale sampling branch. 2. Extended Ablation (03/11-03/20): - Compared MS-Modulated+CA and MS-DSConv+CA under the same evaluation protocol. - Observed improved sensitivity and F1, but reduced precision, suggesting more false-positive pixels. 3. Post-processing Exploration (03/21-03/31): - Started threshold-based false-positive removal experiments. - Explored clDice loss as a possible topology-aware training objective.	2	1 Face-to-face, 1 online/WeChat	4	Group discussion on qualitative results, metric trade-offs and false-positive control.

电子科技大学
格拉斯哥学院
Glasgow College, UESTC

Month	List the main activities (only few words per activity)	Interaction with the supervisor			Any other form of supervisory interaction (second supervisor, industry, fellows etc.)
		Number of meetings	Mode of meeting (face- to-face, online e.g., Skype, WeChat etc.)	Number of emails exchanged	
April	1. Final Experiment Selection (04/01-04/10): - Completed the post-processing threshold comparison and retained X=20 as the final setting. - Tested clDice; it increased sensitivity but reduced precision and F1, so it was not retained. 2. Quantitative and Qualitative Reporting (04/11-04/18): - Generated representative DRIVE overlays and mask comparisons for the final report. - Summarised the final model as MS-DSConv + CA with light post-processing. 3. Final Report Preparation (04/19-04/28): - Expanded the abstract, reference list and result figures after supervisor feedback. - Finalised the cover pages, plagiarism statement, and submission PDFs. - Reported the final F1 improvement from 0.7891 to 0.8027 (+0.0136) as a stable but modest gain.	3	2 Face-to-face, 1 online/WeChat	6	Supervisor feedback on the final report; peer checks on figures, references and PDF formatting.

Appendix: Selected Source Code

Only project-specific code excerpts are included below. Unchanged nnU-Net framework files, datasets, prediction images, and generated outputs are omitted to keep the appendix concise.

A.1 Final Architecture Excerpt

```
# Source: nnunetv2/architecture/pa_net.py (selected final-model excerpt)
# Base PANet, LFAU and CFFN definitions are omitted because they are unchanged framework/support code.
class CoordAtt(nn.Module):
    def __init__(self, inp, oup, reduction=32):
        super(CoordAtt, self).__init__()
        self.pool_h = nn.AdaptiveAvgPool2d((None, 1))
        self.pool_w = nn.AdaptiveAvgPool2d((1, None))
        mip = max(8, inp // reduction)
        self.conv1 = nn.Conv2d(inp, mip, kernel_size=1, stride=1, padding=0)
        # 将 BatchNorm2d 换成 InstanceNorm2d，它对小 batch 极其鲁棒
        self.bn1 = nn.InstanceNorm2d(mip)
        self.act = nn.GELU()
        self.conv_h = nn.Conv2d(mip, oup, kernel_size=1, stride=1, padding=0)
        self.conv_w = nn.Conv2d(mip, oup, kernel_size=1, stride=1, padding=0)


    def forward(self, x):
        n, c, h, w = x.size()
        x_h = self.pool_h(x)
        x_w = self.pool_w(x).permute(0, 1, 3, 2)
        y = torch.cat([x_h, x_w], dim=2)
        y = self.act(self.bn1(self.conv1(y)))
        x_h, x_w = torch.split(y, [h, w], dim=2)
        x_w = x_w.permute(0, 1, 3, 2)
        a_h = self.conv_h(x_h).sigmoid()
        a_w = self.conv_w(x_w).sigmoid()
        return x * a_w * a_h


class MultiScaleSnakeAttention(nn.Module):
    def __init__(self, query_channels, key_channels_list, heads=8, n_points=9): # 建议增至 9 点以体现蛇形连续性
        super().__init__()
        self.heads = heads
        self.n_points = n_points
        self.n_levels = len(key_channels_list)
        self.dim_head = query_channels // heads


        self.value_projs = nn.ModuleList([
            nn.Conv2d(c, query_channels, kernel_size=1) for c in key_channels_list
        ])
```

```
    # 预测序列偏移：不再是独立的 x,y，而是迭代的增量 delta
    self.offset_conv = nn.Conv2d(query_channels, heads * self.n_levels * n_points, kernel_size=3, padding=1)
    self.mask_conv = nn.Conv2d(query_channels, heads * self.n_levels * n_points, kernel_size=3, padding=1)
    self.attn_weights = nn.Conv2d(query_channels, heads * self.n_levels * n_points, kernel_size=3, padding=1)
    self.to_out = nn.Conv2d(query_channels, query_channels, kernel_size=1)


def forward(self, x, feat_list):
    b, c, h, w = x.shape


    # 1. 预测与整理
    delta_offsets = torch.tanh(self.offset_conv(x)) * 0.1
    mask = torch.sigmoid(self.mask_conv(x)).reshape(b, self.heads, self.n_levels, self.n_points, h, w)
    attn = torch.softmax(self.attn_weights(x).reshape(b, self.heads, -1, h, w), dim=2).reshape(b, self.heads, self.n_levels, self.n_points, h, w)


    # 2. 构造蛇形轨迹
    grid_y, grid_x = torch.meshgrid(
        torch.linspace(-1, 1, h, device=x.device, dtype=x.dtype),
        torch.linspace(-1, 1, w, device=x.device, dtype=x.dtype),
        indexing='ij'
    )
    base_grid = torch.stack([grid_x, grid_y], dim=-1).unsqueeze(0) # [1, H, W, 2]


    delta_offsets = delta_offsets.reshape(b, self.heads, self.n_levels, self.n_points, h, w)
    cum_offsets_y = torch.cumsum(delta_offsets, dim=3)
    # 修正：确保类型一致
    cum_offsets_x = torch.arange(self.n_points, device=x.device, dtype=x.dtype).view(1, 1, 1, self.n_points, 1, 1) * (2.0 / w)


    aggregated_feat = 0
    for i, feat in enumerate(feat_list):
        v = self.value_projs[i](feat)
        _, _, v_h, v_w = v.shape


        # === 修正维度不匹配的核心位置 ===
        # 切片后为 5 维，对应的 permute 索引应为 (0, 3, 4, 1, 2)
        level_off_y = cum_offsets_y[:, :, i].permute(0, 3, 4, 1, 2) # (B, H, W, heads, points)
        level_off_x = cum_offsets_x.expand(b, self.heads, self.n_levels, self.n_points, h, w)[:, :, i].permute(0, 3, 4, 1, 2)


        sampling_locs = base_grid.view(1, h, w, 1, 1, 2) + torch.stack([level_off_x, level_off_y], dim=-1)


        # 3. 采样与权重聚合
        v_grouped = v.reshape(b * self.heads, self.dim_head, v_h, v_w).contiguous()
        locs_grouped = sampling_locs.reshape(b * self.heads, h * w, self.n_points, 2).contiguous()
        sampled = F.grid_sample(v_grouped, locs_grouped, mode='bilinear', padding_mode='zeros', align_corners=False)
```

```
            # 对应修正权重张量的维度
            m_attn = (attn[:, :, i] * mask[:, :, i]).permute(0, 1, 3, 4, 2).reshape(b * self.heads, h * w, 1, self.n_points)
            sampled = sampled.permute(0, 2, 3, 1).contiguous()


            aggregated_feat += torch.matmul(m_attn, sampled).reshape(b, self.heads, h, w, self.dim_head).permute(0, 1, 4, 2, 3).reshape(b, c, h, w)


        return self.to_out(aggregated_feat)


# 在 MultiScaleSnakeAttention 类定义之后添加
class EPA_MSDSConv(nn.Module):
    def __init__(self, query_channels, key_channels_list):
        super().__init__()
        # 使用你定义的蛇形版本
        self.ms_snake_attn = MultiScaleSnakeAttention(query_channels, key_channels_list)
        self.dw_conv = nn.Conv2d(query_channels, query_channels, kernel_size=3, padding=1, groups=query_channels)
        self.gelu = nn.GELU()


    def forward(self, x, feat_list):
        # 并行分支融合：蛇形注意力捕捉拓扑 + DW-Conv 捕捉局部细节
        attn_out = self.ms_snake_attn(x, feat_list)
        return self.gelu(attn_out + self.dw_conv(x)) + x


class LPT_MSDSConv_CA(nn.Module):
    def __init__(self, query_channels, key_channels_list):
        super().__init__()
        self.ca = CoordAtt(query_channels, query_channels) # 保留之前表现优异的 CA
        self.lfau = LFAU(query_channels)
        self.epa = EPA_MSDSConv(query_channels, key_channels_list) # 使用蛇形版本
        self.cffn = CFFN(query_channels)


    def forward(self, x, feat_list):
        x = self.ca(x)
        return self.cffn(self.epa(self.lfau(x), feat_list))


class PANet_MSDSConvLPT_CA_NoAFFM(PANet_MSDeformableLPT_CA_NoAFFM):
    def __init__(self, num_classes, input_channels=1, **kwargs):
        super().__init__(num_classes, input_channels, **kwargs)
        features = kwargs.get('features_per_stage', [32, 64, 128, 256])
        # 替换为蛇形 LPT
        self.lpt = LPT_MSDSConv_CA(features[-1], features)
```

A.2 Final nnU-Net Trainer

```
# Source: nnunetv2/training/nnUNetTrainer/nnUNetTrainerPANet_MSDSConvLPT_CA_NoAFFM.py
from nnunetv2.training.nnUNetTrainer.nnUNetTrainerPANet import nnUNetTrainerPANet
from nnunetv2.architecture.pa_net import PANet_MSDSConvLPT_CA_NoAFFM
import torch
from torch.cuda.amp import autocast


class nnUNetTrainerPANet_MSDSConvLPT_CA_NoAFFM(nnUNetTrainerPANet):
    def initialize(self):
        if not self.was_initialized:
            super().initialize()


            # --- 核心稳定性配置 ---
            # 禁用 AMP (混合精度) 以防止 DSConv 累积偏移导致的数值溢出
            self.grad_scaler = None


            print("=" * 60)
            print("SNAKE EXPERIMENT: Multi-Scale MSDSConv + CA (FP32 MODE)")
            print("Target: Break F1-Score 0.80 via Tubular Topology Constraint")
            print("Status: AMP Disabled | Gradient Clipping Active (1.0)")
            print("=" * 60)


            arch_kwargs = self.configuration_manager.network_arch_init_kwargs
            arch_kwargs['deep_supervision'] = True


            # 实例化带蛇形采样与坐标注意力的模型
            self.network = PANet_MSDSConvLPT_CA_NoAFFM(
                input_channels=self.num_input_channels,
                num_classes=self.label_manager.num_segmentation_heads,
                **arch_kwargs
            ).to(self.device)


            self.optimizer, self.lr_scheduler = self.configure_optimizers()


    def train_step(self, batch: dict) -> dict:
        """
        针对蛇形卷积优化的全精度训练步
        """
        x = batch['data']
        y = batch['target']


        x = x.to(self.device, non_blocking=True)
        y = [i.to(self.device, non_blocking=True) for i in y]
```

```
        self.optimizer.zero_grad()

        # 强制走 FP32 逻辑，确保 cumsum 偏移量的精确度
        output = self.network(x)
        l = self.loss(output, y)
        l.backward()

        # 梯度裁剪：防止蛇形轨迹在初期训练时剧烈波动
        torch.nn.utils.clip_grad_norm_(self.network.parameters(), max_norm=1.0)

        self.optimizer.step()

        return {'loss': l.detach().cpu().numpy()}

    def build_network(self):
        """ 推理阶段架构构建 """
        arch_kwargs = self.configuration_manager.network_arch_init_kwargs
        arch_kwargs['deep_supervision'] = False
        return PANet_MSDSConvLPT_CA_NoAFFM(
            input_channels=self.num_input_channels,
            num_classes=self.label_manager.num_segmentation_heads,
            **arch_kwargs
        )
```

A.3 Post-processing and Metric Excerpt

```
# Source: evaluate_metrics_PostProcess.py (selected post-processing and metric excerpt)
from skimage import morphology
from sklearn.metrics import confusion_matrix, roc_auc_score


def apply_area_filter(pred_data, threshold=20):
    if threshold > 0:
        pred_data = morphology.remove_small_objects(
            pred_data.astype(bool), min_size=threshold, connectivity=2
        ).astype(np.uint8)
    return pred_data


def compute_binary_metrics(gt_data, pred_binary):
    tn, fp, fn, tp = confusion_matrix(gt_data, pred_binary, labels=[0, 1]).ravel()
    epsilon = 1e-6
    se = tp / (tp + fn + epsilon)
    sp = tn / (tn + fp + epsilon)
    acc = (tp + tn) / (tp + tn + fp + fn + epsilon)
    pr = tp / (tp + fp + epsilon)
```

```
    f1 = 2 * pr * se / (pr + se + epsilon)
    return {"SE": se, "SP": sp, "ACC": acc, "PR": pr, "F1": f1}


target_thresholds = [0, 20, 30, 50, 90]
```